

Неделя 2: Атрибуты качества

Архитектура ПО / Software Architecture

Цели сегодняшней лекции

К концу лекции вы сможете:

1. Назвать основные **атрибуты качества** (*quality attributes*) и объяснить их простыми словами
2. Показать, что качества **конфликтуют** (компромиссы (*trade-offs*))
3. Написать **сценарий качества** (*quality scenario*) обычным языком (кто, когда, что, с каким результатом)
4. Отличить важные цели от **не-целей** (*non-goals*) («не всё должно быть enterprise grade»)

План (80 минут)

1. Мост от недели 1: структура без «зачем»
2. Шесть атрибутов качества (интуиция + примеры)
3. Конфликты качеств
4. Сценарии обычным языком
5. Не-цели и «достаточно хорошо»
6. Взгляд на практику

Короткие викторины — паузы для проверки, не оценка.

Добавьте новые термины в глоссарий: RU → English → пример.

На прошлой неделе:

- Архитектура (*architecture*) — крупные части и труднооткатываемые решения
- Заинтересованные стороны (*stakeholders*) заботятся о разном

Сегодня: **что значит «хорошая» система** для этих людей — не «красивый код», а измеримые (хотя бы словами) свойства.

Функции vs качества

Функции (*features*)

Что система умеет: войти, сдать домашку, оплатить заказ.

Качества (*qualities*)

Как она это делает: быстро, надёжно, безопасно, удобно для правок.

Запомните

Две системы с одинаковым списком функций могут быть архитектурно совершенно разными — из-за качеств.

На этом курсе держим в голове шесть (достаточно для старта):

По-русски	English
Производительность	performance
Надёжность	reliability
Безопасность	security
Сопровождаемость	maintainability
Удобство использования	usability
Масштабируемость	scalability

Есть и другие (доступность, переносимость. . .) — позже, по мере надобности.

Производительность (*performance*)

- Насколько **быстро** система отвечает и справляется с нагрузкой
- Примеры: страница открывается за 2 с; чат не «висит» при 50 сообщениях в минуту
- Не только «сервер мощный» — и алгоритм, и сеть, и лишние запросы

Частая ошибка

Сказать «сделать быстро» без контекста. Быстро *для кого и при какой нагрузке?*

- Система делает то, что обещала, **достаточно часто** и предсказуемо
- Сбои бывают: важнее, как система себя ведёт *при* сбое
- Примеры: после перезапуска сервера данные не пропали; ошибка оплаты не «съела» заказ дважды

- Кто **имеет право** видеть и менять данные
- Защита от подделки входа, утечек, чужих правок
- Примеры: студент не видит чужие оценки; пароли не хранятся открытым текстом

Архитектурно: где данные, кто к ним ходит, какие границы доверия.

Сопровождаемость (*maintainability*)

- Насколько легко **менять и чинить** систему команде разработчиков
- Ясные модули, понятные связи, меньше «тронь одно — сломалось всё»
- Пример: добавить новый тип уведомлений без переписывания половины приложения

Попробуйте

Вспомните свой учебный проект: что было труднее всего *менять* позже?

Удобство использования (*usability*)

- Насколько людям **понятно и комфортно** пользоваться
- Не только «красивый UI»: тексты ошибок, шаги, доступность
- Пример: сдать домашку с телефона за три понятных шага

Иногда конфликтует с security («ещё один пароль» vs удобный вход).

Масштабируемость (*scalability*)

- Способность справиться с **ростом**: больше пользователей, данных, запросов
- Не всегда нужна «с первого дня» — важно понимать, когда упрётся
- Пример: портал выдерживает неделю экзаменов, а не только обычный вторник

Кто о чём обычно думает

Стейкхолдер	Часто важны
Пользователь	usability, performance, reliability
Разработчик	maintainability
Оператор / IT	reliability, security, scalability
Бизнес / школа	cost + «достаточно» по всем выше

Это не жёсткие правила — стартовая карта для разговора.

Викторина 1 — Какой атрибут?

Викторина

«Студент не должен читать оценки других студентов» — в первую очередь про:

1. Производительность (*performance*)
2. Безопасность (*security*)
3. Масштабируемость (*scalability*)
4. Цвет кнопок

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

2. Безопасность (*security*) — права доступа к данным.

Цвет кнопок — скорее usability/design, не главный здесь атрибут.

Викторина

«Добавить новый отчёт можно за день, не ломая вход и оплату» — ближе всего к:

1. Сопровождаемости (*maintainability*)
2. Только performance
3. Только scalability
4. Не-цели курса

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

1. Сопровождаемость — насколько легко менять систему без каскада поломок.

Нельзя максимизировать всё сразу:

- **Быстро vs дешево:** мощные серверы стоят денег
- **Безопасно vs удобно:** лишние проверки раздражают пользователя
- **Гибко менять vs успеть к дедлайну:** идеальная модульность тоже стоит времени

Архитектура — это выбор **компромиссов** (*trade-offs*), а не поиск «идеала».

Пример конфликта на портале

- Хотим: мгновенный вход без пароля (usability)
- Хотим: никто чужой не увидит оценки (security)

Возможные середины (обсуждаем, не догма):

- Пароль + «запомнить устройство» на своём телефоне
- Вход через школьный аккаунт (один раз настроили — дальше проще)

Таблица компромисса (шаблон)

Выбор	Выигрываем	Теряем / риск
...	...	...
...	...	...

На практике сегодня заполните такую для своего брифа.

Викторина

Какая пара качеств чаще всего конфликтует?

1. Security и usability (строгий вход vs «войти в один клик»)
2. Название переменной и цвет фона слайда
3. Только performance и performance (одно и то же)
4. Качества никогда не конфликтуют

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

1. Security и usability — классический компромисс.

Качества часто тянут систему в разные стороны; задача — выбрать осознанно.

Зачем сценарии?

Размыто: «сделать быстро», «чтобы было безопасно».

Лучше: короткий **сценарий** (*scenario*) — история с проверкой.

Шаблон (достаточно для курса)

Источник: кто или что запускает ситуацию

Стимул: что происходит

Среда: при каких условиях

Ответ системы: что она делает

Мера: как понять, что «ок» (словами или числом)

Пример: нагрузка на портал

«Когда **1000 студентов** открывают портал в **8:55** в день контрольной, страница входа открывается не дольше **3 секунд** у большинства, иначе показываем понятное сообщение, а не пустой экран.» Здесь видны: scalability/performance, надёжность ответа пользователю.

Пример: безопасность оценок

«Когда студент А открывает журнал, он видит **только свои** оценки; попытка открыть URL чужой оценки даёт отказ без утечки данных.» Мера: нет чужих строк в ответе; в логах — факт отказа (для оператора).

- Плохо: «Система должна быть быстрой.»
- Лучше: «Учитель открывает список группы из 40 человек за <2 с на обычном Wi-Fi кампуса.»
- Плохо: «Нужна безопасность.»
- Лучше: «Пароль нельзя прочитывать из базы; сброс пароля только через школьную почту.»

Викторина

Какая формулировка ближе к хорошему сценарию качества?

1. «Сделать портал современным.»
2. «Когда сервер БД недоступен, студент видит сообщение „попробуйте позже“, а уже введённый черновик домашки не теряется на устройстве.»
3. «Использовать микросервисы.»
4. «Добавить больше кнопок.»

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

2. Есть ситуация, ожидаемое поведение и смысл для пользователя (reliability / usability при сбое).

(1) и (3) — размыто или про решение без цели; (4) — не качество.

Не-цели (*non-goals*)

Важно сказать, чего мы **не** обещаем сейчас:

- Не всё должно быть «как у Google / банка»
- Учебный проект может сознательно упростить scalability «на миллион»
- Записать non-goals — честность перед собой и командой

Частая ошибка

Копировать требования из вакансии Senior Architect в курсовую на 3 недели.

- Выберите **топ-3** качества для продукта на этом этапе
- Остальное — «не игнорируем», но не оптимизируем в первую очередь
- Позже (недели про компромиссы и ADR) решения зафиксируем письменно

Викторина 4 — Не-цели (верно/неверно)

Викторина

Верно или неверно?

Для школьного мини-проекта нормально явно написать: «пока не целимся в миллион одновременных пользователей».

1. Верно
2. Неверно — всегда нужно enterprise grade

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

Верно. Явные не-цели помогают не растратить силы и честно обсудить риски.

1. Атрибуты качества описывают *как* система работает, не только *что* умеет
2. Шесть опорных: performance, reliability, security, maintainability, usability, scalability
3. Качества конфликтуют — нужны trade-offs
4. Сценарии превращают лозунги в проверяемые истории
5. Non-goals так же важны, как цели

атрибуты качества (*quality attributes*)

производительность (*performance*) · надёжность (*reliability*) · безопасность (*security*)

сопровождаемость (*maintainability*) · удобство (*usability*) · масштабируемость (*scalability*)

сценарий (*scenario*) · компромисс (*trade-off*) · не-цели (*non-goals*)

Практика (сегодня)

Бриф → топ-3 качества → сценарии → обмен с соседом

Следующая лекция (неделя 3)

Монолит vs модульность (*monolith vs modular*)

Что ещё мутно: названия атрибутов, конфликты или сценарии?